Desafío Técnico: Gestión de Tareas con Spring Boot y Java - Entregable

- La API fue implementada utilizando Java 17, como se especifica en el archivo pom.xml, y se aplicaron características de esta versión, como el uso de Streams y Lambdas.
- Se utilizó Maven como gestor de dependencias, configurado en el archivo pom.xml. En cuanto a la versión de Spring Boot, se usó la más reciente y estable disponible en el momento de la implementación, que es la 3.1.3.
- Para la base de datos, se configuró H2 como base de datos en memoria. Esto se detalla en el archivo application.yml, donde también se habilitó la consola H2, accesible en http://localhost:8081/h2-console/login.do, con las credenciales de usuario spa y contraseña spa123.
- Se crearon tres tablas principales: USERS para usuarios, TASKS para las tareas, y TASK_STATUS para los estados de las tareas. Las tablas USERS y TASK_STATUS contienen datos pre-cargados en el archivo data.sql.
- La capa de persistencia fue implementada utilizando JPA, con repositorios para cada entidad: UserRepository, TaskRepository y TaskStatusRepository.
- La autenticación de usuarios se realiza mediante JWT y asegura que solo los usuarios autenticados puedan acceder a las rutas protegidas. En el archivo JwtUtil, se implementaron métodos para generar, validar y extraer datos del token JWT. El token se genera en el endpoint /auth/login.
- La documentación de la API se generó utilizando OpenAPI y Swagger. El archivo openapi.yml contiene la especificación completa de la API en formato OpenAPI 3.0.1. Esta documentación está disponible en http://localhost:8081/swagger-ui/index.html, donde se pueden probar los endpoints.
- Para la autenticación de usuarios mediante JWT, se implementó el endpoint POST /auth/login en el controlador AuthController. Este recibe un nombre de usuario y contraseña, valida las credenciales y devuelve un token JWT. Además, se incluye el endpoint POST /auth/register para registrar nuevos usuarios.
- Se implementaron todas las operaciones CRUD para la entidad Task en el controlador TaskController. Los métodos son:

    - createTask: Crea una nueva tarea.
    - getTasks: Lista todas las tareas del usuario autenticado.
    - updateTask: Actualiza los datos de una tarea existente.
    - deleteTask: Elimina una tarea por su ID.

- El acceso a las operaciones CRUD de tareas está restringido a usuarios autenticados. las rutas de /tasks requieren un token JWT válido.
- En el repositorio se agregó un archivo tests_postman.json que contiene el conjunto de pruebas necesarias para la API. Este archivo permite cargar todos los endpoints en Postman con datos de prueba predefinidos.
- La implementación cumple con todos los requisitos solicitados. Las funcionalidades están correctamente implementadas, las capas están organizadas conforme a las mejores

prácticas, y se han aplicado principios como la separación de responsabilidades y la inmutabilidad en el código.

- En cuanto a las buenas prácticas, se incluyó un GlobalExceptionHandler para centralizar el manejo de excepciones. Esto mejora la mantenibilidad y asegura respuestas consistentes cuando ocurren errores.
- Finalmente, la API se desarrolló siguiendo un enfoque **API First**, utilizando el archivo openapi.yml como base para definir la estructura y funcionalidades de la API.

Diagramas

Modelo de Base de Datos:

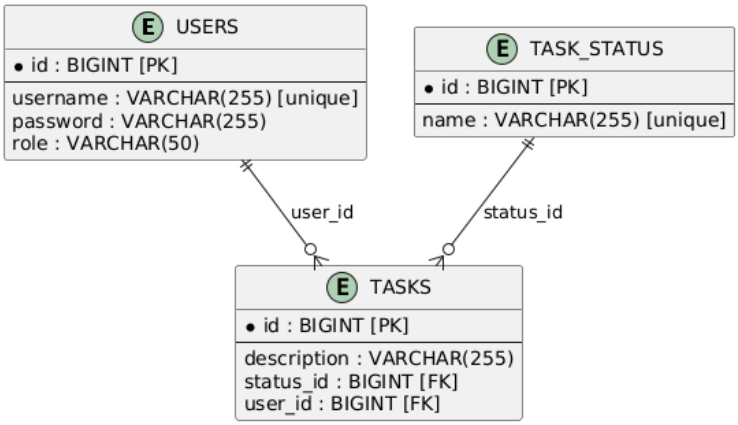
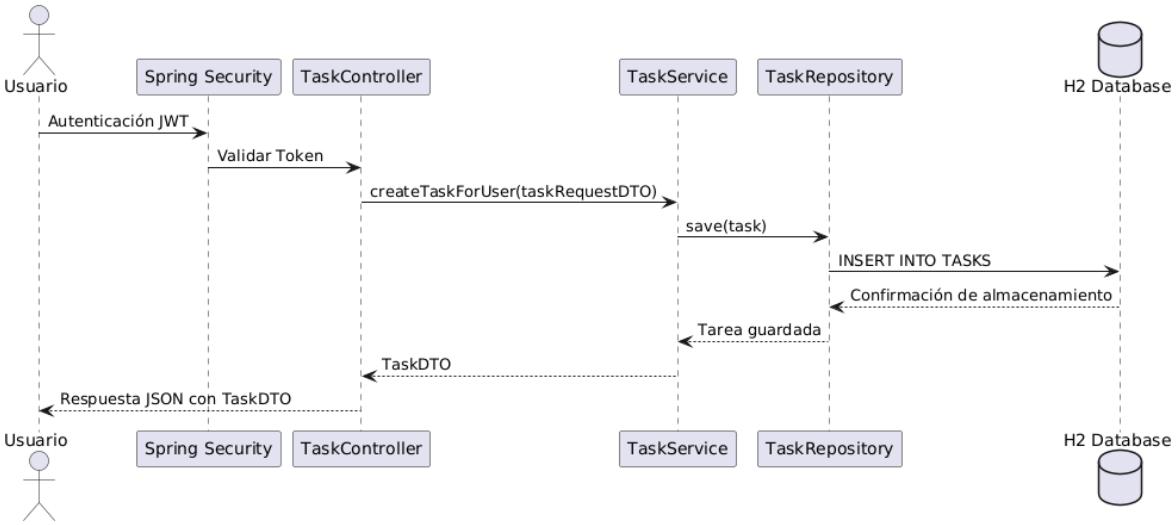


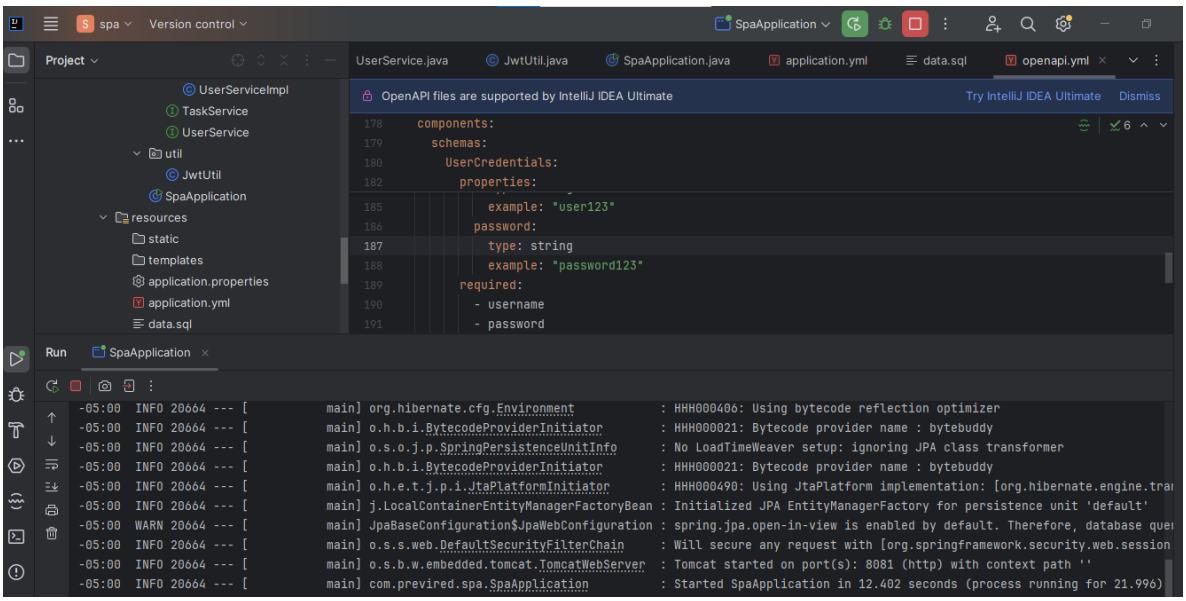
Diagrama de Secuencias:



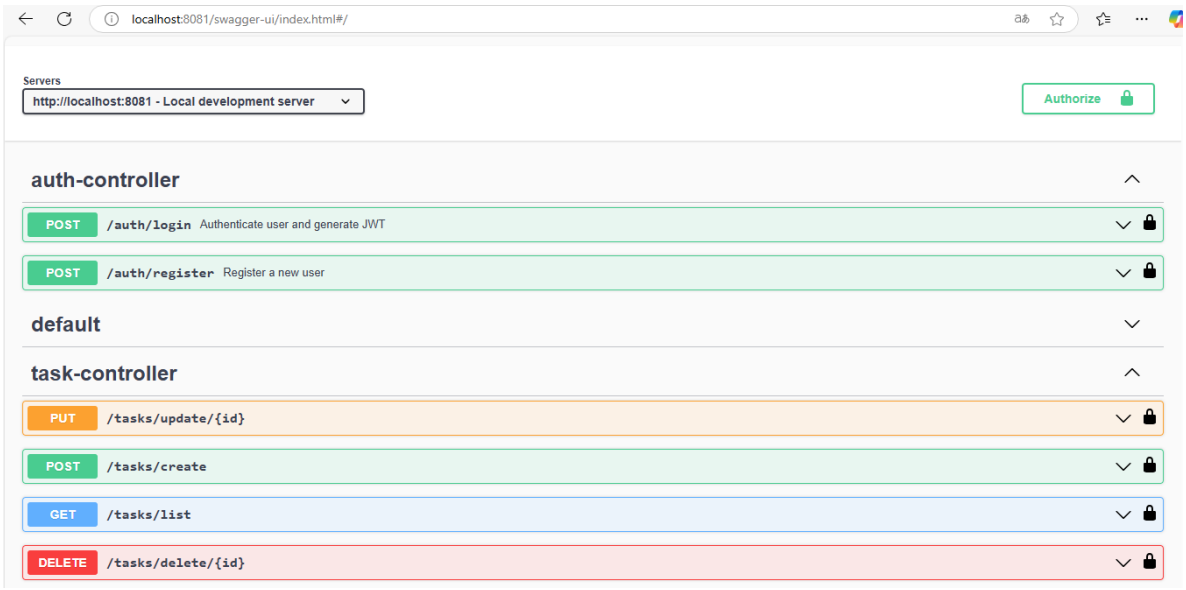
Paso a paso para ejecutar la API:

1. Clonar el repositorio desde https://github.com/previred/desafio-spring-boot.
2. Abrir el proyecto en un IDE compatible, como IntelliJ, con el plugin de Lombok instalado.
3. Ejecutar mvn clean install para descargar las dependencias y compilar el proyecto.

4. Ejecutar la clase principal SpaApplication para levantar el servidor:



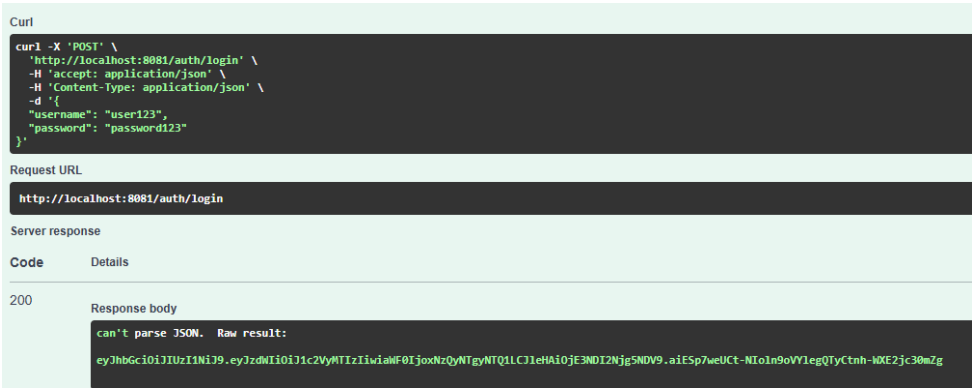
5. Acceder a la URL http://localhost:8081/swagger-ui/index.html para probar los endpoints en Swagger:



Pruebas de endpoints con Swagger y Postman:

- **Autenticación (POST /auth/login)**: Generar un token JWT enviando credenciales válidas, se puede ejecutar desde el inicio con un usuario precargado en la tabla USERS. Por ejemplo:

```
{
 "username": "user123",
 "password": "password123"
}
```



- **Registrar usuario (POST /auth/register)**: Crear un nuevo usuario proporcionando nombre, contraseña y rol. Ejemplo:

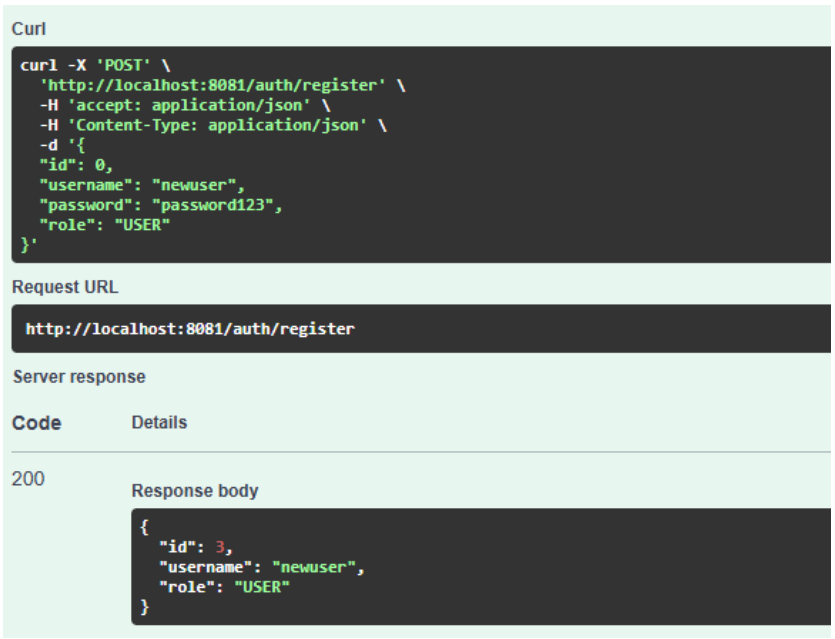
{

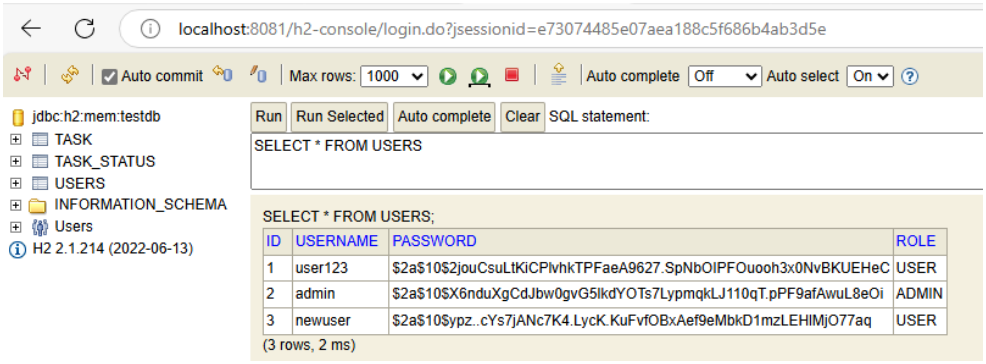
"username": "newuser",

"password": "newpassword123",

"role": "USER"

}





- **Crear tarea (POST /tasks/create)**: Crear una tarea enviando descripción y estado. Ejemplo:

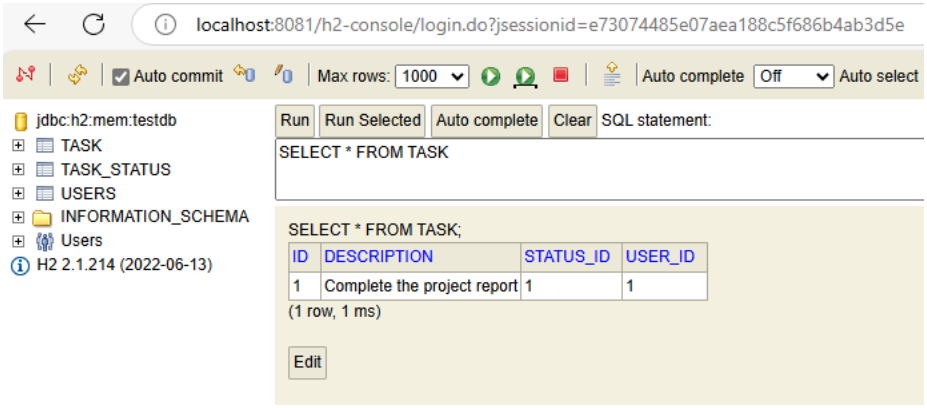
{

"description": "Complete the project report",

"status": "PENDING"

}





- **Listar tareas (GET /tasks/list)**: Devuelve las tareas del usuario autenticado.

Curl

```
curl -X 'GET' \
  'http://localhost:8081/tasks/list' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyMTIzIiwiaWF0IjoxNzQyNTgyODAyLCJleHAiOjE3NDI2NjkyMDJ9.T91T00uF-o1SjQkvvCtC5nwEdL9YgYDBpRxtNimYPyU'
```

Request URL

```
http://localhost:8081/tasks/list
```

Server response

Code	Details
200	Response body

```
[
  {
    "id": 1,
    "description": "Complete the project report",
    "status": "TaskStatus(id=1, name=PENDING)",
    "userId": 1
  }
]
```

- **Actualizar tarea (PUT /tasks/update/{id})**: Actualizar la descripción o estado de una tarea existente.



Curl

```
curl -X 'PUT' \
  'http://localhost:8081/tasks/update/1' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyMTIzIiwiaWF0IjoxNzQyNTgyODAyLCJleHAiOjE3NDI2NjkyMDJ9.T91T00uF-o1SjQkvvCtC5nwEdL9YgYDBpRxtNimYPyU' \
  -H 'Content-Type: application/json' \
  -d '{
  "description": "Complete the project report TEST",
  "status": "PENDING"
}'
```

Request URL

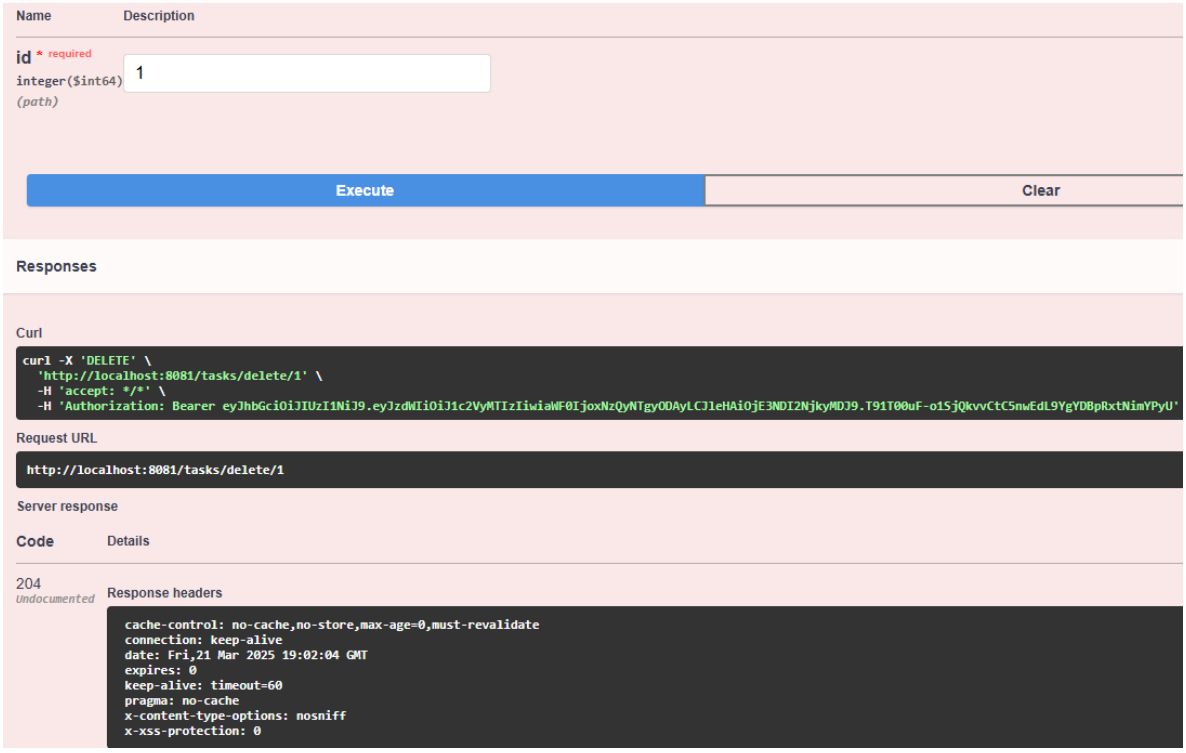
```
http://localhost:8081/tasks/update/1
```

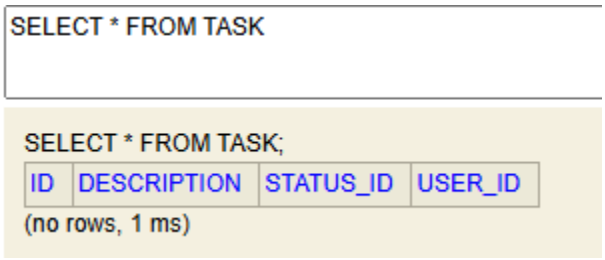
Server response

Code	Details
200	Response body

```
{
  "id": 1,
  "description": "Complete the project report TEST",
  "status": "TaskStatus(id=1, name=PENDING)",
  "userId": 1
}
```

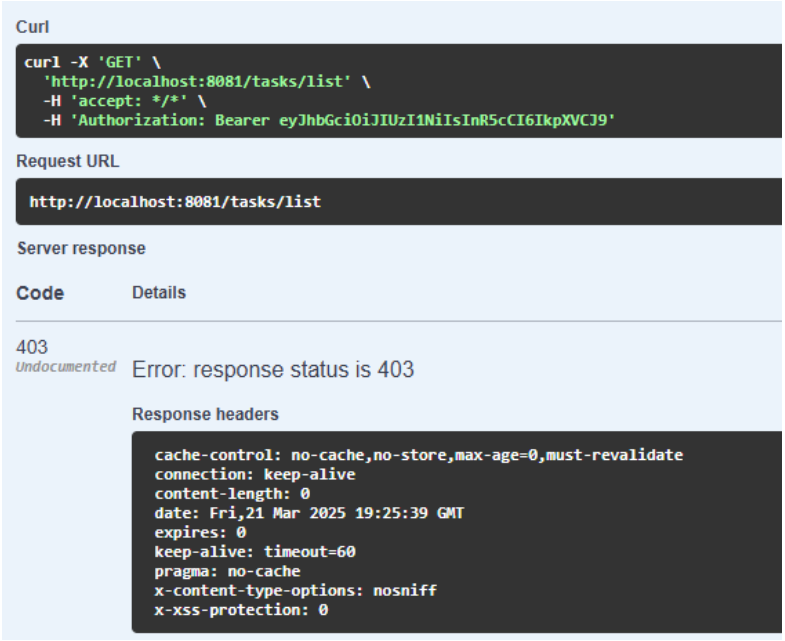
- **Eliminar tarea (DELETE /tasks/delete/{id})**: Eliminar una tarea por su ID.



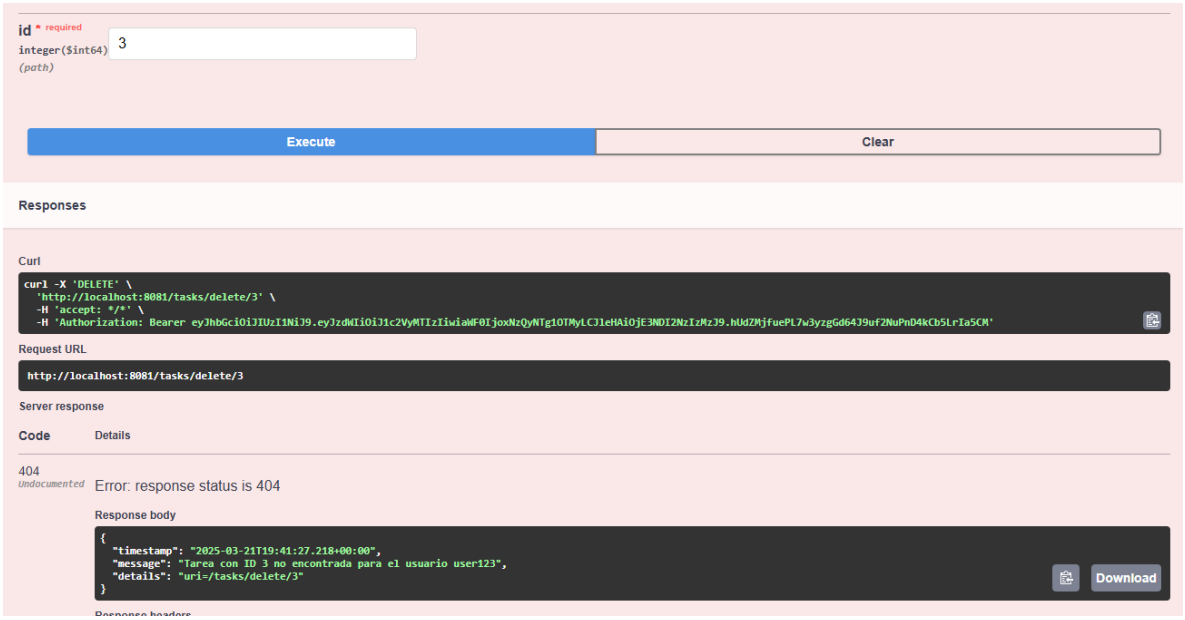


Casos de Excepción:

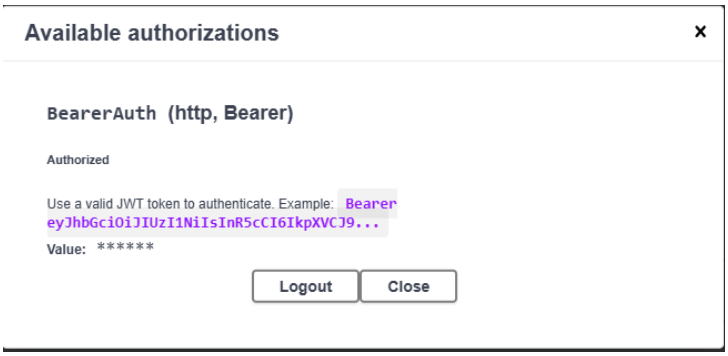
- **Token Inválido o Ausente:** Endpoint: GET /tasks/list Sin token o con un token falso, resultado: error 403.



- **Recurso No Encontrado:** Endpoint: DELETE /tasks/delete/3 (ID inexistente). Resultado: 404 Not Found.



En cada prueba se debe incluir el token JWT en el header de autorización como Bearer <token>. En la interface de swagger se debe autenticar con el token en el botón “Authorize”:



En postman se debe agregar en el header Authorization:

